

PYTHON3 REQUESTS PLAYBOOK

Chapter 1 — HTTP, APIs & The Requests Library

A comprehensive, production-focused guide for aspiring Data Scientists & AI Engineers

SECTION 1 — THEORY

1.1 What Is a Playbook Course?

A playbook course is a learning format centered on real-world problems and reusable solutions rather than abstract theory. Instead of walking through every possible edge case, it distills the most common workflows and patterns into concise, actionable examples you can immediately copy, adapt, and deploy.

For data scientists and AI engineers, this approach is especially valuable because the field moves fast. You rarely have time to re-derive first principles — you need working patterns you can reach for quickly. That is the core philosophy of this playbook.

Key characteristics of this format:

- Emphasizes practical, real-world workflows over academic theory.
- Provides reusable code patterns you can drop into any project.
- Covers the most common tasks developers encounter day to day.
- Designed for hands-on, iterative learning — read, run, modify, repeat.

1.2 The World Wide Web (WWW)

The World Wide Web is a globally distributed system of interconnected resources — web pages, documents, images, videos, APIs, and more — all accessible through the internet. It is important to distinguish between the Internet (the underlying physical network of cables, routers, and servers) and the WWW (a service that runs on top of that network).

When you open a browser and navigate to a URL, several things happen in sequence:

- Your browser (the client) reads the URL and extracts the domain name.
- A DNS (Domain Name System) lookup translates the domain name into an IP address.
- The browser establishes a connection to the target server.
- An HTTP or HTTPS request is sent, and the server responds with the requested content.
- The browser renders the response — HTML, CSS, JavaScript, images — into what you see on screen.

Every resource on the web is identified by a Uniform Resource Locator (URL). A URL has a specific anatomy:

Anatomy of a URL	
<code>https://api.example.com:443/v1/users?page=1&limit=10#section</code>	
<code>https</code>	→ Protocol (HTTP or HTTPS)
<code>api</code>	→ Subdomain

example.com	→ Domain
:443	→ Port (default for HTTPS; usually omitted)
/v1/users	→ Path (the specific resource)
?page=1	→ Query parameters (key=value pairs)
#section	→ Fragment (client-side anchor, not sent to server)

1.3 HTTP — HyperText Transfer Protocol

HTTP is the application-layer protocol that governs how messages are exchanged between clients and servers on the web. It is stateless — every request is independent and carries all the information the server needs to respond. The server does not retain any memory of previous requests by default.

HTTP defines a simple but powerful communication model:

- A client (browser, Python script, mobile app) sends a request.
- The server receives, processes, and returns a response.
- The response includes a status code, response headers, and optionally a body.

💡 HTTP vs HTTPS

HTTPS is simply HTTP with an encrypted transport layer (TLS/SSL). All data in transit is encrypted, preventing eavesdropping and man-in-the-middle attacks. Modern APIs exclusively use HTTPS. You should never send credentials or sensitive data over plain HTTP.

1.4 HTTP Methods

HTTP defines a set of request methods (also called verbs) that indicate the desired action to be performed on a resource. Choosing the correct HTTP method is fundamental to designing and consuming RESTful APIs correctly.

Two important properties define the behavior of HTTP methods:

- **Safe:** The method does not modify server state. GET is safe; POST, PUT, DELETE are not.
- **Idempotent:** Calling the method multiple times produces the same result as calling it once. GET, PUT, and DELETE are idempotent; POST is not.

Method	Description	Safe?	Idempotent?
GET	Retrieve data from the server. Does not modify any resource.	Yes	Yes
POST	Submit new data to the server to create a resource.	No	No
PUT	Replace an existing resource entirely with new data.	No	Yes
PATCH	Partially update an existing resource.	No	No

DELETE	Remove a resource from the server permanently.	No	Yes
--------	--	----	-----

1.5 Client-Server Architecture

The client-server model is the foundational architectural pattern of the web. It separates concerns cleanly: the client is responsible for the user interface and initiating requests, while the server is responsible for data storage, business logic, and returning responses.

In Python-based data science and AI work, the client is usually your Python script, a Jupyter notebook, or an automated pipeline. The server is typically a REST API endpoint — for example, an OpenAI API, a weather data provider, or your own deployed FastAPI service.

A client can be:

- A web browser (Chrome, Firefox) loading a web page.
- A mobile application making API calls on behalf of a user.
- A Python script using the requests library to fetch or post data.
- A microservice communicating with another microservice in a distributed system.

1.6 The Request-Response Cycle

Every interaction on the web follows the same fundamental pattern: a request goes out, a response comes back. Understanding this cycle in detail is critical because every bug you encounter — timeouts, 401 errors, empty responses — maps to a specific step in this cycle.

The full cycle unfolds as follows:

- Step 1 — The client constructs a request, specifying a method (GET, POST, etc.), a URL, optional headers, and an optional body.
- Step 2 — The request is transmitted over the network to the server.
- Step 3 — The server receives the request, authenticates and authorizes it if necessary, and executes the appropriate business logic.
- Step 4 — The server constructs a response containing a status code, response headers, and a body (often JSON).
- Step 5 — The client receives the response, checks the status code, and processes the body.

 Key Insight for Data Scientists

When you call an ML model API or a data provider API in production, you are always in the client role. Understanding exactly what goes into each step helps you write resilient code that handles errors, retries on transient failures, and processes responses efficiently.

1.7 Anatomy of an HTTP Request

An HTTP request is a structured message sent by the client to the server. It consists of four parts:

HTTP Request Structure
GET /v1/users?page=1 HTTP/1.1
Host: api.example.com
Authorization: Bearer eyJhbGciOiJIUzI1NiJ9...
Accept: application/json
User-Agent: python-requests/2.31.0
[Optional Request Body – used with POST/PUT/PATCH]

Breaking this down:

- **Request Line:** Contains the HTTP method (GET), the resource path (/v1/users), and the HTTP version.
- **Headers:** Key-value metadata pairs. Common ones include Host, Authorization, Content-Type, Accept, and User-Agent.
- **Query Parameters:** Appended to the URL after a ?, these are key=value pairs used to filter or paginate data (e.g., ?page=1&limit=10).
- **Request Body:** Present in POST, PUT, and PATCH requests. Contains the data to send, typically as JSON.

1.8 Anatomy of an HTTP Response

When the server processes a request, it returns an HTTP response. Like the request, it has a defined structure:

HTTP Response Structure
HTTP/1.1 200 OK
Content-Type: application/json
Content-Length: 512
Cache-Control: no-cache
Date: Sat, 20 Jun 2026 10:00:00 GMT
{
"id": 1,
"name": "Ada Lovelace",
"email": "ada@example.com"
}

The response contains:

- **Status Line:** HTTP version and the status code with its reason phrase (e.g., 200 OK, 404 Not Found).
- **Response Headers:** Metadata about the response, such as Content-Type (what kind of data is being returned) and Cache-Control (how long to cache it).
- **Response Body:** The actual content — usually JSON when working with REST APIs. Can also be HTML, plain text, XML, or raw binary data.

1.9 HTTP Status Codes — Complete Reference

Status codes are three-digit numbers that tell the client what happened with its request. They are grouped into five categories by their first digit:

- **1xx** — Informational: Request received, processing continues.
- **2xx** — Success: The request was received, understood, and accepted.
- **3xx** — Redirection: Further action is needed to complete the request.
- **4xx** — Client Error: The request contains bad syntax or cannot be fulfilled.
- **5xx** — Server Error: The server failed to fulfill a valid request.

Status Code	Meaning & When You See It
200 OK	Request succeeded. Data is returned in the response body.
201 Created	Resource was successfully created by a POST request.
204 No Content	Request succeeded but no body is returned (common for DELETE).
301 Moved Permanently	Resource has permanently moved to a new URL.
400 Bad Request	Server could not understand the request due to malformed syntax.
401 Unauthorized	Authentication is required and has failed or not been provided.
403 Forbidden	Client is authenticated but does not have permission.
404 Not Found	The requested resource does not exist on the server.
429 Too Many Requests	Rate limit exceeded. Slow down your requests.
500 Internal Server Error	Server encountered an unexpected error.
503 Service Unavailable	Server is temporarily unavailable, usually due to maintenance.

1.10 Application Programming Interfaces (APIs)

An API is a contract between two software systems that defines how they communicate. In the context of web development and data science, a web API exposes data or functionality over HTTP so that any client — regardless of programming language or platform — can access it.

Why APIs matter for data scientists and AI engineers:

- Data ingestion: Most real-world data pipelines begin with an API call — to a financial data provider, social media platform, weather service, or IoT device stream.
- Model serving: Your trained ML models are most commonly exposed as APIs (using FastAPI or Flask) so other services can consume predictions.
- Third-party AI services: OpenAI, Anthropic, HuggingFace, and virtually every major AI provider expose their capabilities through REST APIs.

REST (Representational State Transfer) is the dominant architectural style for modern web APIs. A REST API:

- Uses standard HTTP methods (GET, POST, PUT, DELETE) to perform operations.
- Returns data — almost universally JSON — in a structured, predictable format.
- Is stateless: each request is self-contained.
- Organizes resources around nouns (e.g., /users, /models, /predictions), not verbs.

1.11 The Python Requests Library

The requests library is the de facto standard for making HTTP calls in Python. It abstracts away the complexity of urllib and the raw socket layer, giving you a clean, human-friendly API that handles encoding, redirects, authentication, sessions, and SSL verification automatically.

Installation is straightforward via pip:

Installation

```
pip install requests
```

Core capabilities of the requests library:

- Supports all HTTP methods: GET, POST, PUT, PATCH, DELETE, HEAD, OPTIONS.
- Handles query parameters, request headers, and request bodies cleanly.
- Automatically decodes JSON responses via the .json() method.
- Supports authentication: Basic Auth, Bearer tokens, OAuth, Digest Auth.
- Manages cookies and sessions with the Session object.
- Handles SSL/TLS certificate verification out of the box.
- Supports file uploads, streaming responses, and timeouts.

1.12 Working with JSON Responses

JSON (JavaScript Object Notation) is the lingua franca of modern APIs. It is a lightweight, human-readable data format that maps naturally to Python's built-in data structures: JSON objects become Python dictionaries, and JSON arrays become Python lists.

When a server returns JSON, the requests library gives you two ways to access it:

- `response.text`: Returns the raw JSON as a Python string. You would then need to call `json.loads()` on it manually.
- `response.json()`: Parses the JSON string automatically and returns a Python dictionary or list. This is almost always what you want.

Important

`response.json()` will raise a `json.JSONDecodeError` if the response body is not valid JSON. Always check `response.status_code` before calling `.json()` to ensure the request actually succeeded.

1.13 Cookies and Sessions

HTTP is inherently stateless — every request is independent. Cookies are the mechanism that adds state to this stateless protocol. A cookie is a small piece of data set by the server (via the Set-Cookie header) and stored by the client, which automatically includes it in subsequent requests to the same domain.

Common uses for cookies:

- Authentication: Storing a session token so the user stays logged in across page loads.
- Personalization: Remembering user preferences (language, theme).
- Tracking: Identifying a returning user (analytics and advertising).

A Session in the requests library is an object that persists certain parameters across multiple requests made to the same host. This means:

- Cookies set by the server are automatically stored and re-sent with every subsequent request.
- HTTP connections are reused (connection pooling), reducing latency on repeated calls.
- Common headers or authentication credentials can be set once on the session, rather than repeated in every call.

1.14 Authenticating API Requests

Most production APIs require authentication — a way for the server to verify who is making a request and whether they are authorized to access the requested resource. The requests library supports several authentication mechanisms.

Common authentication strategies:

Authentication Strategies Overview

```
# 1. API Key in Header (most common for data APIs)
```

```
headers = {"X-API-Key": "your_api_key"}
```

```
# 2. Bearer Token (OAuth 2.0, JWT)
headers = {"Authorization": "Bearer YOUR_ACCESS_TOKEN"}

# 3. Basic Auth (username + password)
response = requests.get(url, auth=("username", "password"))

# 4. OAuth 2.0 Token (via requests-oauthlib)
from requests_oauthlib import OAuth2Session
oauth = OAuth2Session(client_id, token=token)
response = oauth.get(url)
```

1.15 OAuth 2.0 — Delegated Authorization

OAuth 2.0 is the industry-standard framework for delegated authorization. Instead of sharing your password with a third-party application, OAuth 2.0 allows you to grant that application limited access to your resources (on another service) via an access token.

You encounter OAuth 2.0 every time you click 'Sign in with Google' or 'Connect with GitHub'. The key insight is that you (the Resource Owner) grant access to a Client Application without ever sharing your password.

OAuth 2.0 defines four roles:

- Resource Owner: The user who owns the data and grants access.
- Client Application: The third-party app requesting access (your Python script or web app).
- Authorization Server: The server that issues access tokens (e.g., Google's OAuth server).
- Resource Server: The API server that holds the protected data (e.g., Google Drive API).

The Authorization Code Flow (most secure for server-side apps):

- Step 1: Your app redirects the user to the Authorization Server's login page.
- Step 2: The user logs in and grants permission via a consent screen.
- Step 3: The Authorization Server redirects back to your app with a short-lived authorization code.
- Step 4: Your app exchanges the authorization code for a long-lived access token (server-to-server, never exposed to the browser).
- Step 5: All subsequent API calls include the access token in the Authorization header.

1.16 HTTPS, SSL/TLS, and Certificate Verification

HTTPS (HTTP Secure) is HTTP with an encrypted transport layer using TLS (Transport Layer Security), the successor to SSL. When you connect to an HTTPS endpoint, a TLS handshake takes place before

any data is exchanged. This handshake authenticates the server's identity and establishes an encrypted channel.

How TLS works at a high level:

- The server presents its SSL/TLS certificate, issued by a trusted Certificate Authority (CA).
- Your client (requests library) verifies the certificate against its built-in list of trusted CAs.
- If the certificate is valid, a symmetric encryption key is negotiated.
- All subsequent communication is encrypted with that key.

The requests library verifies SSL certificates by default (`verify=True`). This is the correct and safe behavior. The following table summarizes the SSL options available:

verify Parameter	Behavior	When to Use
<code>verify=True</code> (default)	Verifies cert against trusted CA bundle	Always — in production
<code>verify=False</code>	Disables certificate verification entirely	Only local testing — never in production
<code>verify='/path/to/ca.pem'</code>	Verifies against a custom CA bundle	Enterprise environments with private CAs
<code>cert=('c.crt', 'k.key')</code>	Sends a client certificate with the request	Mutual TLS (mTLS) authentication

SECTION 2 — PRACTICAL & CODING (IPYTHON NOTEBOOK WALKTHROUGH)

This section walks through every code cell of the accompanying Jupyter notebook. Each cell is presented exactly as you would see it in a notebook environment, followed by a detailed explanation of what the code does, why it is written that way, and what you should observe when you run it.

How to Follow Along

Open a Jupyter notebook or a Python script in your editor. Run each cell in order. Observe the output. Then read the explanation to understand what happened. Modifying the code and re-running is the fastest way to internalize these concepts.

Cell 1 — Verifying Your First HTTP Request

In [1]:

```
import requests

response = requests.get("https://api.github.com")
print(response.status_code)
```

Explanation:

This is the simplest possible HTTP interaction with the requests library, and it is deliberately minimal so you can confirm that everything — your internet connection, your Python environment, and the requests library — is working correctly.

Line 1: `import requests` loads the entire requests library into your current Python session. This is all you need to start making HTTP calls.

Line 3: `requests.get()` sends an HTTP GET request to the GitHub API root endpoint. This endpoint is publicly accessible without any authentication — it is specifically designed as a health-check and discovery endpoint.

Line 4: `response.status_code` is an integer property on the Response object. A value of 200 confirms that the server received the request, processed it successfully, and returned a response. If you see anything other than 200 — for example 403 (Forbidden) or 503 (Service Unavailable) — it signals a problem to investigate.

Expected output: 200

Cell 2 — Exploring the Raw Response Body

In [2]:

```
import requests

response = requests.get("https://httpbin.org/get")
```

```
print(response.text)
```

Explanation:

httpbin.org is a freely available HTTP testing service maintained by the community. It is invaluable for learning and debugging because it echoes back exactly what it received — your headers, your IP address, your query parameters — as a JSON response.

The /get endpoint accepts GET requests and returns a JSON object containing the complete details of your incoming request. This is perfect for inspecting how your requests look from the server's perspective.

response.text returns the response body as a Python string. For a JSON endpoint like this one, response.text is the raw JSON string before any parsing. You will see something like:

```
{  "args": {},  "headers": {    "Accept": "*/*",    "Accept-Encoding": "gzip, deflate, br",    "Host": "httpbin.org",    "User-Agent": "python-requests/2.31.0"  },  "url": "https://httpbin.org/get" }
```

Notice the User-Agent header — requests automatically adds this to every request, identifying itself as the client. This is standard behaviour and you can override it by passing a custom headers dictionary.

Cell 3 — Inspecting Status Codes and Response Headers

In [3]:

```
import requests

response = requests.get("https://httpbin.org/get")
print(response.status_code)
print(response.headers)
```

Explanation:

This cell demonstrates accessing two critical parts of the HTTP response: the status code and the response headers.

response.status_code (printed first): As explained in Cell 1, this is 200 for a successful request. In production code, you should always check this before accessing the response body. A common pattern is: if response.status_code == 200: ... else: handle_error().

response.headers (printed second): This is a dictionary-like object (specifically a CaseInsensitiveDict) containing all the HTTP headers returned by the server. Common headers you will encounter include:

Content-Type: tells you the format of the response body (e.g., application/json; charset=utf-8).

Content-Length: the size of the response body in bytes.

Server: identifies the server software (e.g., nginx, Apache, gunicorn).

Cache-Control: instructs clients and proxies how to cache the response.

Understanding response headers is especially important when debugging API issues — they often reveal crucial information about rate limiting (X-RateLimit-Remaining), authentication failures, or redirect loops.

Cell 4 — Sending Query Parameters with GET

In [4]:

```
import requests

response = requests.get(
    "https://httpbin.org/get",
    params={"name": "John"}
)
print(response.url)
```

Explanation:

Query parameters are the mechanism for filtering, searching, or paginating data in GET requests. They are appended to the URL as key=value pairs after a ? character, with multiple parameters separated by &.

Instead of manually constructing the URL string (which is error-prone and does not handle special characters correctly), requests provides the params argument, which accepts a Python dictionary. The library automatically encodes the dictionary into a properly formatted query string.

response.url prints the final URL that was actually sent, including the encoded query string. In this case, you will see: `https://httpbin.org/get?name=John`

This is a particularly useful debugging technique. If your API call is returning unexpected results, printing response.url immediately reveals whether your parameters were encoded correctly. For example, if you pass `params={'q': 'hello world'}`, requests correctly encodes the space as %20, resulting in `?q=hello%20world`.

Real-world example: When calling a data API with pagination, you would pass `params={'page': 2, 'limit': 50, 'sort': 'created_at'}` to fetch the second page of 50 results sorted by creation date.

Cell 5 — Checking a Specific Status Code

In [5]:

```
import requests

response = requests.get("https://httpbin.org/status/200")
print(response.status_code)
```

Explanation:

The `/status/{code}` endpoint on httpbin.org is a simulator: it returns whatever HTTP status code you specify in the URL path. This makes it ideal for testing how your error-handling code behaves against different scenarios without needing a real API that produces those errors.

In this cell, we request status code 200, so the response.status_code will be 200. The response body is empty (no content is returned), which is realistic — many endpoints return a status code with no body.

Try modifying the URL to test other status codes: `/status/404`, `/status/500`, `/status/401`. Observe how the `response.status_code` changes. This is how you should build and test your error-handling logic before deploying to production.

Best practice: In production code, use `response.raise_for_status()` instead of manually checking `status_code`. This method automatically raises an `HTTPError` exception for any 4xx or 5xx status code, which you can then catch with a `try/except` block.

Cell 6 — Consuming a REST API and Parsing JSON

In [6]:

```
import requests

response = requests.get(
    "https://jsonplaceholder.typicode.com/users"
)
data = response.json()
print(data[0]["name"])
```

Explanation:

JSONPlaceholder is a free fake REST API for testing and prototyping. It provides realistic-looking data (users, posts, comments, todos) without requiring any authentication or setup.

This cell demonstrates the complete, practical workflow for consuming a REST API in a data pipeline:

Step 1 — Make the request: `requests.get()` sends a GET request to the `/users` endpoint.

Step 2 — Parse the response: `response.json()` parses the JSON response body into a Python list of dictionaries (since the API returns an array of user objects).

Step 3 — Access the data: `data[0]` accesses the first user object (a Python dictionary), and `["name"]` retrieves the value for the 'name' key.

The output will be the name of the first user returned by the API, for example: Leanne Graham.

This exact pattern — get, parse, access — is what you will use hundreds of times when building data ingestion pipelines. The data variable is now a regular Python list that you can iterate over, convert to a pandas DataFrame, or pass to further processing functions.

Cell 7 — Fetching a Single Resource by ID

In [7]:

```
import requests

response = requests.get(
    "https://jsonplaceholder.typicode.com/posts/1"
)
```

```
data = response.json()
print(data["title"])
```

Explanation:

This cell demonstrates retrieving a specific resource by its ID — one of the most fundamental REST API patterns. In REST, resources are addressed by their path: `/posts` returns all posts, while `/posts/1` returns only the post with ID 1.

The `response.json()` call here returns a Python dictionary (not a list), because we are fetching a single object. The dictionary has keys like `id`, `title`, `body`, and `userId`.

Printing `data["title"]` extracts just the title field. You will see something like: `sunt aut facere repellat provident occaecati excepturi optio reprehenderit`.

This pattern scales to any REST API. For example, to fetch details of a specific GitHub repository, you would call `requests.get('https://api.github.com/repos/pandas-dev/pandas')` and then parse the JSON to extract metadata like the star count, description, or open issues.

Cell 8 — Accessing Response Content (text, bytes, JSON)

In [8]:

```
import requests

response = requests.get(
    "https://jsonplaceholder.typicode.com/users/1"
)
print(response.json())
```

Explanation:

This cell illustrates the three ways to access the content of an HTTP response, and when to use each:

`response.text`: Returns the response body decoded as a UTF-8 (or the encoding declared in Content-Type) Python string. Use this when you need the raw text — for example, when scraping HTML or handling a plain-text response.

`response.content`: Returns the response body as raw bytes (a bytes object in Python). Use this when downloading binary files such as images, PDFs, or zip archives. You would then write the bytes to a file with `open('file.pdf', 'wb')` as `f: f.write(response.content)`.

`response.json()`: The most convenient option for API work. It calls `json.loads()` on `response.text` internally and returns a Python dictionary or list. Use this for virtually all API calls that return JSON.

In this cell, we call `response.json()` on a single user endpoint, which prints the complete user dictionary. Explore the structure of the output — you will see keys like `id`, `name`, `username`, `email`, `address`, `phone`, `website`, and `company`.

Cell 9 — Working with Cookies

In [9]:

```
import requests

response = requests.get(
    "https://httpbin.org/cookies/set/user/John"
)

print(response.cookies)
```

Explanation:

The `/cookies/set` endpoint on `httpbin.org` simulates a server setting a cookie. When you call this URL with `/key/value` in the path (here: `/user/John`), the server responds with a `Set-Cookie` header that instructs the client to store `user=John`.

`response.cookies` is a `RequestsCookieJar` object — a dictionary-like container that holds all cookies set by the server in this response. Printing it shows something like: `<RequestsCookieJar[Cookie(name='user' value='John' ...)]>`.

Important behaviour: By default, `requests` does NOT automatically follow the redirect and re-send the cookie. The `/cookies/set` endpoint redirects you to `/cookies` (which shows stored cookies), but with a plain `requests.get()` call, the cookie is captured but not automatically sent back on the redirect. To handle cookies across redirects and multiple requests correctly, you need a `Session` object — which is covered in the next cells.

Cell 10 — Using Sessions for Persistent Cookies

In [10]:

```
import requests

session = requests.Session()
response = session.get(
    "https://httpbin.org/cookies/set/user/John"
)

print(session.cookies.get_dict())
```

Explanation:

A `requests.Session` object is the correct tool whenever you need to make multiple requests to the same server — especially for login-protected APIs, web scraping, or any workflow where authentication state must be maintained.

`requests.Session()` creates a session instance. Unlike individual `requests.get()` calls, the session maintains state between requests. This means:

- Cookies received in any response are automatically stored in `session.cookies`.

- Those stored cookies are automatically sent with every subsequent request to the same domain.

- TCP connections are reused (connection pooling), reducing overhead on repeated calls.

`session.cookies.get_dict()` converts the cookie jar to a plain Python dictionary for easy inspection. Expected output: `{'user': 'John'}`.

Production use case: When automating login to a web application, you would call `session.post(login_url, data={'username': ..., 'password': ...})`. The server returns a session cookie in the response. From that point on, every call through the same session object automatically sends that cookie, keeping you authenticated.

Cell 11 — Simulating a Login with Sessions

In [11]:

```
import requests

session = requests.Session()
login_data = {
    "username": "admin",
    "password": "secret"
}
session.post("https://httpbin.org/post", data=login_data)
```

Explanation:

This cell demonstrates the session-based login workflow — arguably the most important pattern for any scraping or automation task involving authenticated web apps.

A dictionary `login_data` is constructed with the credentials. In a real application, these would come from environment variables (using `os.getenv()`) or a secrets manager — never hardcoded in source code.

`session.post()` sends an HTTP POST request. POST is the correct method for submitting form data or creating resources. The `data=` parameter tells requests to encode the dictionary as `application/x-www-form-urlencoded` (the format web browsers use for HTML form submissions).

After this call, the session object has stored any cookies the server returned (typically a session ID token). All subsequent calls through this session — `session.get(dashboard_url)`, `session.get(data_url)` — will include those cookies automatically, simulating being logged in.

Note: `httpbin.org/post` simply echoes back what it receives without actually authenticating anything. In a real workflow, you would replace this URL with your target application's login endpoint.

Cell 12 — Bearer Token Authentication

In [12]:

```
import requests

headers = {"Authorization": "Bearer ACCESS_TOKEN"}
response = requests.get(
    "https://api.example.com/data",
```

```
headers=headers
)
```

Explanation:

Bearer token authentication is the most common authentication mechanism for modern REST APIs, including virtually every AI service API you will encounter as a data scientist (OpenAI, Anthropic, HuggingFace, etc.).

The Authorization header carries the credentials. Its value follows the format: Bearer <token>, where <token> is a long, opaque string (often a JWT — JSON Web Token) issued by the API provider.

In practice, you obtain the token by logging in via the API provider's website or by exchanging your credentials through their OAuth 2.0 token endpoint. You then store the token securely (as an environment variable) and load it at runtime:

```
import os token = os.getenv("OPENAI_API_KEY") headers = {"Authorization": f"Bearer {token}"}
```

The headers= parameter in requests.get() accepts a dictionary of headers to include with the request. The requests library merges these with its own default headers (like User-Agent and Accept-Encoding).

This cell uses a placeholder URL and token. Replace ACCESS_TOKEN with your real token and api.example.com with the actual API endpoint to make it functional.

Cell 13 — HTTPS Request and SSL Verification

In [13]:

```
import requests

response = requests.get(
    "https://www.google.com"
)

print(response.status_code)
```

Explanation:

This cell makes a basic HTTPS request to Google to confirm that SSL/TLS verification works correctly in your environment. The requests library, by default, verifies the server's SSL certificate against the system's trusted CA bundle (provided by the certifi package, which is installed as a dependency of requests).

If the certificate is valid and trusted, the request proceeds normally and you get a 200 status code. If the certificate is invalid, expired, or self-signed (and not in your trusted store), requests raises an SSLError before even sending the request.

The fact that this works silently and correctly — without any SSL-related configuration on your part — is one of the key advantages of the requests library over lower-level alternatives.

Google returns HTTP 200 for its homepage. The response body would be the full HTML of the Google homepage, but we are only printing the status code here.

Cell 14 — Explicit SSL Verification (verify=True)

In [14]:

```
import requests

response = requests.get(
    "https://www.weather.com/my-town",
    verify=True
)
```

Explanation:

Passing `verify=True` is redundant (it is the default), but it is excellent practice to include it explicitly in production code. Making the SSL verification behaviour explicit in your codebase means anyone reading your code immediately understands that security is intentional, not accidental.

This is especially important in team environments or when your code will be reviewed, audited, or maintained by others. Code that silently relies on defaults can be risky — if someone later changes the default or copies the pattern without the default behaviour, security regressions can go unnoticed.

The `verify` parameter accepts three types of values: `True` (verify with default CA bundle), `False` (disable verification — dangerous), or a string path to a custom CA bundle file. The next two cells explore the latter two cases.

Cell 15 — Disabling SSL Verification (Testing Only)

In [15]:

```
import requests

response = requests.get(
    "https://www.weather.com/my-town",
    verify=False
)
```

Explanation:

Setting `verify=False` completely disables SSL certificate verification. The `requests` library will also emit a `InsecureRequestWarning` to alert you that this is happening. The warning looks like: `InsecureRequestWarning: Unverified HTTPS request is being made to host 'www.weather.com'.`

When should you use this? The only legitimate scenario is testing against a local development server or a staging environment that uses a self-signed certificate that is not registered with a CA. For example, if you spin up a local FastAPI server with a self-signed cert for testing, you would use `verify=False` to avoid SSL errors during development.

When should you absolutely NOT use this? Never in production. Never when making requests that involve authentication credentials, personal data, or sensitive information. Disabling SSL verification opens your application to man-in-the-middle attacks, where an attacker can intercept and modify your requests and responses.

If you need to suppress the warning during testing, you can call: `requests.packages.urllib3.disable_warnings()`. But treat this as a temporary measure only.

Cell 16 — Custom CA Bundle

In [16]:

```
import requests

response = requests.get(
    "https://example.com",
    verify="/path/to/ca_bundle.pem"
)
```

Explanation:

In enterprise environments, companies often operate internal services secured with certificates signed by a private, company-managed Certificate Authority (CA) — one that is not in the public trusted CA list. A typical example is an internal data API or machine learning model serving endpoint behind a corporate firewall.

In this scenario, passing `verify=True` would raise an `SSLError` (because the public CA bundle does not know about your company's private CA), but passing `verify=False` would be insecure. The correct solution is to pass the path to your company's CA bundle file (a `.pem` file containing the trusted certificate chain).

The CA bundle is typically provided by your IT or security team. You store it somewhere accessible in your deployment environment (for example, at `/etc/ssl/certs/company_ca.pem` or as a mounted secret in a Kubernetes pod).

Replace `/path/to/ca_bundle.pem` with the actual path to the `.pem` file in your environment. The requests library will then use this file to verify the server's certificate.

Cell 17 — Client Certificates (Mutual TLS)

In [17]:

```
import requests

response = requests.get(
    "https://example.com",
    cert=("client.crt", "client.key")
)
```

Explanation:

Standard TLS only authenticates the server (the client verifies the server's certificate). Mutual TLS (mTLS) goes one step further: the server also verifies the client's identity by requiring the client to present its own certificate.

This is the `cert` parameter in requests — it accepts a tuple of (`certificate_file`, `private_key_file`) paths. The certificate file (`.crt`) contains the client's public certificate, and the private key file (`.key`) contains the corresponding private key used to prove ownership of the certificate.

Where is mTLS used? It is common in high-security enterprise APIs, financial services, healthcare data APIs, and microservice-to-microservice communication in zero-trust architectures. If an API requires mTLS, the API provider will issue you both the `.crt` and `.key` files.

Security note: The private key file (`.key`) must be kept confidential. It should be stored in a secrets manager (AWS Secrets Manager, HashiCorp Vault) and never committed to version control. Use file permissions (`chmod 600`) to restrict access on the filesystem.

Chapter Summary — Key Patterns Reference

The following table consolidates the most important code patterns from this chapter into a quick-reference cheat sheet:

Task	Code Pattern
Simple GET request	<code>requests.get(url)</code>
GET with query params	<code>requests.get(url, params={'key': 'value'})</code>
Parse JSON response	<code>data = response.json()</code>
Check status code	<code>response.status_code</code>
Add auth header	<code>requests.get(url, headers={'Authorization': 'Bearer TOKEN'})</code>
Use a session	<code>s = requests.Session(); s.get(url)</code>
POST form data	<code>requests.post(url, data={'key': 'value'})</code>
POST JSON body	<code>requests.post(url, json={'key': 'value'})</code>
Custom CA bundle	<code>requests.get(url, verify='/path/to/ca.pem')</code>
Client certificate	<code>requests.get(url, cert=('c.crt', 'k.key'))</code>
Disable SSL (dev only)	<code>requests.get(url, verify=False)</code>
Raise on error	<code>response.raise_for_status()</code>

💡 What's Next

In the next chapter, we go deeper: advanced POST requests with JSON payloads, error handling and retry strategies, pagination through multi-page APIs, authentication with OAuth 2.0 using the `requests-oauthlib` library, and building a complete data ingestion pipeline that pulls from a live API into a pandas DataFrame.

